

Abstraction at What Cost? Evaluating Python DSLs for Transformer GPU Kernel Primitives on Nvidia Blackwell

Tanmay Nandanikar^[0009-0000-1351-1262] and Bartosz Sawicki^[0000-0003-1832-759X]

Warsaw University of Technology, Poland
{tanmay.nandanikar.dokt,bartosz.sawicki}@pw.edu.pl

Abstract. Writing high-performance GPU kernels for transformer workloads remains a significant engineering challenge. Nvidia’s cuTile library, introduced in December 2025, offers a tile-centric programming model that aims to simplify kernel authoring while retaining near-expert performance. In this work, we evaluate cuTile against Triton, Nvidia Warp, and optimized baselines (PyTorch) across three representative transformer primitives: general matrix multiplication (GEMM), Flash Attention, and Paged Attention. Various batch sizes and sequence lengths were compared for GEMM, and Flash Attention respectively. All experiments are conducted on an Nvidia Blackwell GPUs (Blackwell sm_120) and profiled with Nvidia Nsight Systems. We find that cuTile delivers competitive kernel-level performance on GEMM and Flash Attention, but needs careful optimization to close the gap with Triton on Paged Attention. Optimizations include compile-time index tiling, disabling gather bounds checking, memory latency hints, target architecture selection, optimization hints, and changes in math expressions. Our results suggest that cuTile’s tile abstraction is well-suited for regular, compute-bound kernels but faces challenges on memory-bound workloads. We discuss the implications for GPU kernel productivity and outline directions for improvement.

Keywords: GPU kernels · transformer inference · CUDA · Triton · cuTile · performance benchmarking · Blackwell architecture

1 Introduction

For two decades, the Single-Instruction Multiple-Thread (SIMT) model has defined the GPU programming landscape, forcing developers to micromanage thousands of threads, explicit synchronization barriers, and brittle shared memory layouts. However, as silicon complexity scales (culminating in the Nvidia Blackwell architecture (B200/B300) and its specialized Tensor Cores) the burden of manual thread-level orchestration has become a fundamental bottleneck to both developer productivity and peak hardware utilization. We are witnessing a

mandatory strategic shift toward tile-based abstractions. In this paradigm, computation is expressed as operations on multidimensional data blocks (tiles) rather than scalar threads. This transition is essential for abstracting hardware intricacies, allowing the compiler and runtime to handle resource allocation while ensuring that kernels remain forward-compatible across evolving architectural generations.

The transformer architecture [1] underpins modern large language models (LLMs) and vision transformers. As models scale to billions of parameters, the latency of individual GPU kernels (matrix multiplications, attention mechanisms, and KV-cache management) becomes a critical bottleneck. Writing efficient CUDA kernels requires deep expertise in shared memory tiling, warp-level primitives, register pressure management, and occupancy tuning.

Several Python-embedded domain-specific languages (DSLs) lower this barrier. OpenAI’s Triton [2] provides a block-level programming model with automatic memory coalescing. Nvidia Warp [6] offers a tile-based API for simulation and general GPU computation. Most recently, Nvidia introduced `cuda.tile` (cuTile) [6] in December 2025: a tile-centric Python DSL that wraps CUDA’s Cooperative Groups and Tensor Memory Accelerator (TMA) hardware. CuTile’s `ct.load/ct.store/ct.mma` primitives abstract away memory staging and tensor core invocation, while `ct.gather` provides indirect memory access. This raises a natural question: *does this abstraction sacrifice performance, and if so, where?*

We present one of the first independent evaluations of cuTile alongside Triton, Warp (for GEMM), and baseline implementations (cuBLAS/PyTorch SDPA) across three transformer primitives on Blackwell hardware:

1. **GEMM**: the compute backbone of all transformer layers.
2. **Flash Attention** [3]: fused, memory-efficient self-attention for prefill.
3. **Paged Attention** [5]: KV-cache attention with non-contiguous memory pages for autoregressive decoding.

Beyond wall-clock performance, we provide deep profiling of occupancy, shared memory usage, and kernel launch overhead. We additionally benchmark Paged Attention at two precisions (FP32 and BF16) to separate memory-bandwidth bottlenecks from compute and staging bottlenecks. Our contributions are:

- A head-to-head comparison of cuTile, Triton, Warp (only for GEMM), and PyTorch across three transformer primitives at multiple problem sizes on Blackwell.
- A follow-up cuTile variant study showing that the original Paged Attention kernel is not performance-representative: compile-time index generation, disabled gather bounds checks, and occupancy hints improve BF16 latency at CTX=4096 from 5,517 μ s to 609 μ s, within 3.3% of Triton.
- A specialized indirect tile-extraction microbenchmark showing that cuTile `ct.gather` remains $1.16\times$ slower than Triton’s block-pointer/TMA path on random 32×32 block extraction, indicating residual overhead for irregular access even after the larger Paged Attention gap is removed.

- A cross-precision analysis showing that the *baseline* cuTile Paged Attention kernel scales much more weakly from FP32 to BF16 than Triton, motivating the variant study and highlighting the sensitivity of memory-bound kernels to implementation details.

2 Background and Related Work

Evolution of GPU DSLs. To abstract the complexities of modern GPU hardware, several domain-specific languages (DSLs) and compilers have been developed. Early frameworks like Halide [9] focused on image processing pipelines, while TVM [8] introduced automated end-to-end compilation for deep learning tensors. For dense linear algebra, low-level template libraries such as CUTLASS [6] provide high performance but require deep C++ expertise. Recently, Python-embedded DSLs have emerged to bridge the gap between high-level framework usability and low-level hardware utilization.

Triton. Triton [2] is an open-source Python DSL and compiler for GPU programming. Programs specify operations on fixed-size blocks; the compiler handles memory coalescing, shared memory allocation, and warp scheduling via an MLIR-based backend. Triton is the default kernel backend in PyTorch 2.x through `torch.compile`.

Nvidia Warp. Warp [6] provides `tile_load/tile_matmul/tile_store` with a kernel-per-thread model extended by cooperative tile primitives. It targets differentiable simulation but supports general GPU computation. Warp does not support BF16.

cuTile (`cuda.tile`). CuTile [6], released December 2025, wraps TMA hardware and tensor cores in a high-level tile API. It provides `ct.load/ct.store` for contiguous tile I/O routed through TMA, `ct.mma` for tensor-core GEMM, and `ct.gather` for indirect memory access. Nvidia’s TileGym [6] repository provides reference cuTile kernels for GEMM, Flash Attention, and contiguous decode attention. They notably do not provide a Paged Attention implementation with non-contiguous KV caches.

GPU Benchmarking Studies for Deep Learning. Historically, GPU benchmarking focused on raw hardware microbenchmarks [10,11] or end-to-end suites like MLPerf [12], which obscure kernel-level DSL overheads. While early Triton evaluations [2,14] demonstrated near-cuBLAS performance for compute-dense operations on Volta and Ampere, generative AI has fundamentally shifted bottleneck profiles. Prior literature lacks a systematic evaluation of how high-level tile abstractions perform on Hopper and Blackwell’s asynchronous hardware (TMA) when handling the highly irregular, memory-bound access patterns that dominate modern LLM inference. This work directly addresses that gap.

Transformer Inference Primitives. Modern transformer models [1] rely on three critical kernel primitives that serve as primary targets for DSL benchmarks. GEMM and FlashAttention [3,16] are heavily compute-bound, testing a DSL’s ability to maximize Tensor Core occupancy and hide memory latency. Conversely, autoregressive decoding relies on Paged Attention [5], which stores Key-Value (KV) caches in non-contiguous physical memory pages to manage dynamic sequences. This transforms attention into a severely memory-bandwidth-bound operation with indirect, irregular memory accesses. While libraries like FlashInfer [17] optimize these patterns using specialized C++, evaluating how Python DSLs handle such indirect accesses (via cuTile’s `ct.gather` or Triton’s pointer arithmetic) remains an open challenge that forms the core of our evaluation.

3 Evaluation

We evaluate cuTile on three representative GPU workloads: dense matrix multiplication (GEMM), Flash Attention, and Paged Attention with irregular memory access.

3.1 Experimental Setup

All experiments run on an Nvidia GeForce RTX 5070 Ti (Blackwell, GB203, sm_120) with 70 SMs, 16 GB GDDR7 (896 GB/s bandwidth), and 48 MB L2 cache. The software stack consists of PyTorch 2.10.0, Triton 3.6.0, `cuda-tile` 1.1.0, Warp 1.11.1, and CUDA Toolkit 13.1.

Kernel Implementations. We implement custom kernels in cuTile, Triton, and Warp where applicable, while PyTorch dispatches to vendor libraries (cuBLAS or SDPA). Unless otherwise noted, kernels operate on FP16 inputs and follow standard warmup and benchmarking procedures.

Benchmarking Protocol and Correctness. Performance is measured via Nsight Systems ($n \geq 5$ invocations) and CUDA events, following standard warmup iterations. Each configuration is run three times to confirm reproducibility (cross-run variability $< 1.2\%$, coefficient of variation $< 1.1\%$). All kernels are validated against a PyTorch reference. In FP32, cuTile and Triton produce bitwise-identical outputs for Paged Attention; in BF16, maximum absolute differences are $\leq 7.8 \times 10^{-3}$, consistent with BF16 rounding.

3.2 GEMM

We evaluate square $N \times N$ FP16 matrices. CuTile uses tile sizes (64, 64, 32); Triton uses (128, 256, 64); Warp uses (64, 64, 32). PyTorch dispatches to cuBLAS.

Table 1 presents FP16 GEMM latencies. CuTile consistently outperforms both Triton and Warp, remaining within $1.3\times$ of cuBLAS. At $N=4096$, cuTile achieves $1,809\mu\text{s}$ versus cuBLAS’s $1,407\mu\text{s}$ ($1.29\times$), while Triton requires

Table 1. GEMM kernel latency (μs , mean $\pm \sigma$) on FP16 square matrices ($n=5$).

N	cuBLAS	cuTile	Triton	Warp
512	4.5 ± 0.1	5.4 ± 0.04	64.7 ± 0.5	19.6 ± 0.1
1024	26.7 ± 0.2	35.1 ± 0.2	126.0 ± 0.9	106.1 ± 0.1
2048	183 ± 1.6	238.7 ± 0.4	586 ± 3.9	731 ± 0.6
4096	$1,407 \pm 4.2$	$1,809 \pm 1.5$	$4,639 \pm 18$	$5,617 \pm 2.6$
8192	$11,171 \pm 10$	$14,132 \pm 15$	$35,077 \pm 34$	$46,070 \pm 51$

Table 2. Flash Attention latency (μs , mean $\pm \sigma$) at various sequence lengths ($B=2$, $H=8$, $D=64$, FP16, $n=25$).

Seq Len	PyTorch SDPA	cuTile	Triton
512	15.8 ± 0.0	19.7 ± 0.1	21.9 ± 0.0
1024	54.3 ± 0.1	69.8 ± 0.2	65.4 ± 0.4
2048	204.7 ± 0.2	262.0 ± 0.4	245.4 ± 0.6
4096	802.0 ± 0.6	956.8 ± 0.7	944.0 ± 0.9
8192	$2,990 \pm 3.2$	$3,775 \pm 4.5$	$3,489 \pm 2.9$

4,639 μs ($3.29\times$) and Warp 5,617 μs ($3.99\times$). CuTile’s `ct.mma` maps directly to TMA-based data movement and tensor core execution, adding negligible overhead over the vendor library for this regular, compute-bound workload.

3.3 Flash Attention

Inputs are shaped ($B=2, H=8, S, D=64$) in FP16. CuTile uses tiles (64, 32); Triton uses blocks (128, 64). PyTorch dispatches to its vendor Flash Attention backend.

Table 2 reports Flash Attention latencies in μs (Nsight Systems kernel time). PyTorch SDPA, dispatching to a vendor-optimized kernel, is fastest at all sequence lengths. Sequence length is referred to as S henceforth. CuTile and Triton perform within 7% of each other, with the gap to SDPA widening at longer sequences ($1.26\times$ at $S=8192$). This is expected: SDPA uses architecture-specific tile shapes and software pipelining. CuTile is faster than Triton at $S=512$ (19.7 vs. 21.9 μs), suggesting that cuTile’s lower launch overhead or smaller block size is advantageous for short sequences. All Flash Attention measurements have coefficient of variation $< 0.6\%$.

BF16 Flash Attention latencies are within 0.1% of the FP16 numbers for all frameworks, consistent with the compute-bound nature of this workload: halving the data width does not change tensor-core throughput.

3.4 Paged Attention

We use $B=16$, $H=16$, $D=128$, block size 16, and context lengths from 512 to 16,384. Benchmarks are performed in FP32 and BF16.

Table 3. Paged Attention latency (μs , mean $\pm \sigma$) in FP32 for the *baseline* cuTile kernel ($B=16$, $H=16$, $D=128$; $n=1,011$ for fused kernels).

CTX	PyTorch (Ref) [†]	cuTile	Triton
512	1,235	852 ± 1.2	154 ± 0.4
1024	2,221	$1,717 \pm 1.5$	305 ± 0.7
2048	3,600	$3,372 \pm 2.9$	610 ± 1.1
4096	7,015	$6,665 \pm 3.7$	$1,212 \pm 2.2$
8192	14,047	$13,329 \pm 5.7$	$2,479 \pm 4.1$
16384	28,037	$26,693 \pm 7.3$	$5,090 \pm 10.1$

[†]PyTorch unfused: σ unavailable (per-iteration timing aggregates $\sim 161\text{K}$ sub-kernel launches); cross-run variability $< 1.2\%$.

The baseline cuTile kernel uses `ct.gather` for indirect KV-cache access, whereas Triton performs equivalent access via pointer arithmetic. PyTorch iterates over batch elements in Python.

Table 3 and Figure 1 show the behavior of the *baseline* cuTile Paged Attention kernel. In FP32, Triton outperforms this baseline by $5.2\text{--}5.5\times$ across all context lengths, while baseline cuTile is only marginally faster than the unfused PyTorch reference. At CTX=4096, Triton achieves $1,212 \pm 2.2 \mu\text{s}$, baseline cuTile $6,665 \pm 3.7 \mu\text{s}$, and PyTorch $7,015 \mu\text{s}$. The stability of this gap initially suggests a severe problem on the cuTile path, motivating the follow-up variant study in Table 4. All fused-kernel measurements have coefficient of variation $< 0.3\%$ ($n=1,011$).

Paged Attention Optimized Variants Table 4 revisits Paged Attention at a representative operating point (CTX=4096, BF16). The tuned cuTile variant removes auxiliary index-tensor loads by generating index tiles in-kernel using compile-time ranges, eliminates gather bounds checks when indices are known valid, and introduces explicit latency hints on memory operations to better overlap DRAM accesses with computation. These changes remove unnecessary global-memory traffic and reduce indexing overhead in the inner loop.

The aggressive variant keeps the same paged, non-contiguous KV-cache layout but further adds explicit kernel configuration hints (target occupancy and compiler optimization level) together with refined latency annotations for key loads and stores, improving scheduling and memory-level parallelism on Blackwell GPUs.

Together these changes reduce cuTile latency from $5,517 \mu\text{s}$ to $1,114 \mu\text{s}$ (tuned) and then to $609 \mu\text{s}$ (aggressive), a $9.05\times$ improvement over the original kernel and within 3.3% of Triton. Because the data structure and access pattern remain paged, this result shows that the original gap was not an intrinsic consequence of non-contiguous KV-cache access.

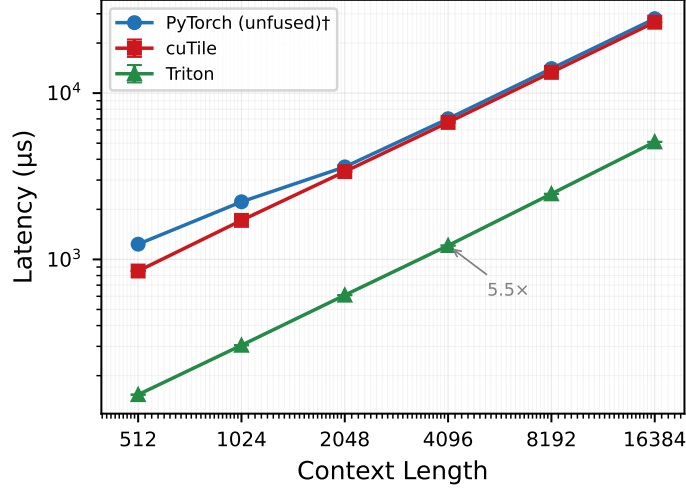


Fig. 1. Paged Attention latency vs. context length (FP32, log scale). Error bars show $\pm 1\sigma$ ($n=1,011$); bars are smaller than markers. [†]PyTorch unfused: per-iteration σ unavailable.

Table 4. Representative BF16 Paged Attention variant study at CTX=4096 ($B=16$, $H=16$, $D=128$).

Kernel	Latency (μ s)	Max Diff vs. PyTorch
PyTorch (Ref.)	3,291.54	—
cuTile (old)	5,517.06	1.465×10^{-3}
cuTile (tuned)	1,113.90	2.075×10^{-3}
cuTile (aggressive)	609.44	2.075×10^{-3}
Triton	589.84	1.465×10^{-3}

3.5 Specialized Indirect Tile Extraction

To isolate irregular block selection outside the softmax/reduction pipeline, we extract 16,384 random 32×32 tiles from an 8192×8192 matrix. Table 5 shows that Triton’s block-pointer/TMA kernel remains faster than cuTile’s `ct.gather` version, but only by $1.16\times$. This is a meaningful difference, yet far smaller than the original baseline Paged Attention gap, indicating that indirect access overhead remains real but does not by itself explain the earlier $5\times$ – $9\times$ disparity.

4 Analysis

In this section, we investigate the underlying hardware and software factors driving the performance differences observed in our evaluation. We use the roofline model to classify workloads, analyze kernel occupancy to explain the `ct.gather`

Table 5. Specialized indirect tile-extraction benchmark (16,384 random 32×32 tiles from an 8192×8192 FP16 matrix).

Kernel	Latency (μ s)
PyTorch	2,124,311.56
cuTile (<code>ct.gather</code>)	136.62
Triton (block pointer / TMA)	117.40

bottleneck, examine cross-precision scaling behavior, and highlight the overheads of dynamic kernel selection and dispatch in unfused baselines.

4.1 Roofline Model

The roofline model [7] bounds performance by the minimum of peak compute throughput π and memory bandwidth β scaled by operational intensity I (FLOPs/byte). The ridge point $I^* = \pi/\beta$ separates memory-bound ($I < I^*$) from compute-bound ($I > I^*$) regimes. Based on our hardware specifications and the peak cuBLAS GEMM result, the RTX 5070 Ti achieves an observed FP16 throughput of $\pi = 98.4$ TFLOPS. With $\beta = 896$ GB/s, the ridge point is $I^* \approx 110$ FLOPs/byte.

At a representative problem size of 4096, both FP16 GEMM ($I_{\text{GEMM}} = N/3 \approx 1,365$) and Flash Attention ($I_{\text{Flash}} = S/2 = 2,048$) operate far to the right of the ridge point ($> 12 \times I^*$), making them heavily compute-bound. In contrast, decoding a single query token in Paged Attention yields a constant operational intensity of $I_{\text{Paged}} = 2/b$ (where b is the element byte width). This results in $I = 0.5$ for FP32 and $I = 1.0$ for BF16—over $100\times$ below the ridge point, placing Paged Attention firmly in the memory-bound regime.

Results on the roofline. Figure 2 plots the achieved performance of each framework against the roofline at representative size 4096. GEMM and Flash Attention cluster in the compute-bound region: cuBLAS reaches 97.7 TFLOPS (99% of the observed peak) and all DSLs exceed 29 TFLOPS. The *baseline* Paged Attention kernels occupy the opposite extreme. Triton’s fused kernel achieves 443 GFLOPS at $I=0.5$ (FP32), which is 98.9% of the memory-bandwidth ceiling ($\beta \cdot I = 448$ GFLOPS)—an optimally bandwidth-utilized memory-bound kernel. In BF16, Triton reaches 901 GFLOPS at $I=1.0$ (ceiling: 896 GFLOPS), confirming near-ideal scaling with reduced element width. The baseline cuTile kernel, by contrast, achieves only 80.7 GFLOPS in FP32 (18.0% of the ceiling) and 95.8 GFLOPS in BF16 (10.7%), showing that the original implementation leaves most of the available bandwidth unused. Importantly, the follow-up variant study in Table 4 demonstrates that this shortfall is not an inherent roofline limit of cuTile itself, but of a particular paged-attention formulation.

This roofline classification provides a unifying lens for our results: cuTile’s tile abstraction is effective for compute-bound kernels that lie far to the right of the ridge point, where tensor-core utilization is the dominant performance factor.

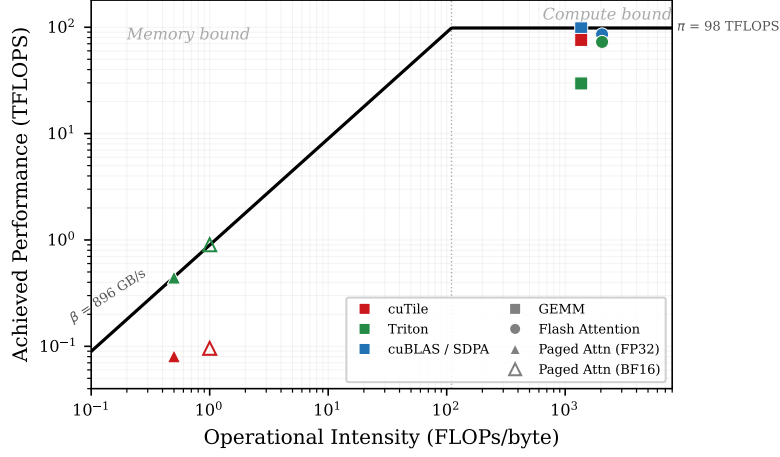


Fig. 2. Roofline analysis on the RTX 5070 Ti ($\pi=98.4$ TFLOPS, $\beta=896$ GB/s). Filled markers denote FP16/FP32; open markers denote BF16. GEMM and Flash Attention are compute-bound ($I \gg I^*$). Paged Attention is deeply memory-bound ($I \ll I^*$): Triton reaches 99% of the bandwidth ceiling, while cuTile reaches only 18%.

For memory-bound kernels to the left of the ridge point, where efficient global-memory access patterns determine throughput, performance becomes much more sensitive to auxiliary staging, index handling, and scheduling choices on the `ct.gather` path.

4.2 Occupancy and the `ct.gather` Bottleneck

Table 6 presents per-kernel resource consumption at a representative size of 4096. For GEMM and Flash Attention, all frameworks use high register counts (255 registers/thread), yielding modest occupancy (8–17%). Performance is still competitive because these workloads are compute-bound: tensor cores stay busy even at low occupancy.

Paged Attention tells a different story. The baseline cuTile kernel uses 168 registers/thread and 75 KB of shared memory per block, limiting each SM to a single resident block (25% occupancy). Triton uses only 56 registers and 512 bytes of shared memory, fitting ~ 9 blocks per SM (75% occupancy)—a $3\times$ occupancy advantage.

These numbers implicate the original `ct.gather`-based formulation, but the follow-up variant study shows that `ct.gather` alone is not the full explanation. The baseline kernel also loads auxiliary index tensors from memory and gives the compiler no gather bounds or occupancy hints. Removing those overheads and adding explicit occupancy guidance reduces CTX=4096 BF16 latency from 5,517 μ s to 609 μ s while preserving the same paged, non-contiguous KV-cache layout. Thus the original result should be read as a warning about naive imple-

Table 6. Kernel resource usage and estimated occupancy at size 4096.

Bench.	Kernel	Regs	Thr.	Warps	Smem	Occ.
GEMM	cuTile	255	256	8	41 KB	16.7%
	Triton	255	128	4	96 KB	8.3%
	cuBLAS	96	128	4	49 KB	16.7%
	Warp	128	128	4	25 KB	16.7%
Flash	cuTile	255	256	8	33 KB	16.7%
	Triton	255	128	4	49 KB	8.3%
	SDPA	255	128	4	49 KB	8.3%
Paged	cuTile	168	384	12	75 KB	25.0%
	Triton	56	128	4	512 B	75.0%

Table 7. Paged Attention latency (μ s, mean $\pm \sigma$) in BF16 ($B=16$, $H=16$, $D=128$; $n=1,011$ for fused kernels).

CTX	PyTorch (Ref) [†]	cuTile	Triton
512	2,202	706 ± 0.8	64 ± 0.7
1024	2,191	$1,422 \pm 1.6$	153 ± 0.7
2048	2,206	$2,819 \pm 2.0$	297 ± 1.0
4096	3,357	$5,603 \pm 2.6$	596 ± 1.7
8192	7,163	$11,231 \pm 3.8$	$1,219 \pm 3.7$
16384	14,184	$22,590 \pm 7.5$	$2,488 \pm 4.3$

[†]See Table 3 footnote.

mentations of cuTile on irregular workloads, not proof that paged attention is fundamentally beyond the programming model.

TileGym’s [6] reference decode-attention kernels still provide an important clue: they use `ct.load` with `allow_tma=True` on contiguous KV caches, whereas the paged variant must rely on `ct.gather`. The specialized indirect-access benchmark in Table 5 shows that this gather path remains somewhat costlier than Triton’s block-pointer/TMA route even in isolation, but only by 16%.

4.3 Cross-Precision Scaling: FP32 vs. BF16

To further diagnose the bottleneck, we benchmark Paged Attention in BF16. For a memory-bandwidth-bound kernel, halving the data width from 32-bit to 16-bit should yield $\sim 2\times$ speedup. Table 7 and Figure 3 reveal three distinct scaling regimes.

Triton: $\sim 2\times$ (*expected*). Triton’s Paged Attention kernel is memory-bandwidth-bound: it loads Q, K, V from global memory via pointer arithmetic. Halving the data width from FP32 to BF16 halves the memory traffic, yielding speedups of 1.98–2.37 $\times$ across context lengths. This is the textbook result for a bandwidth-limited kernel.

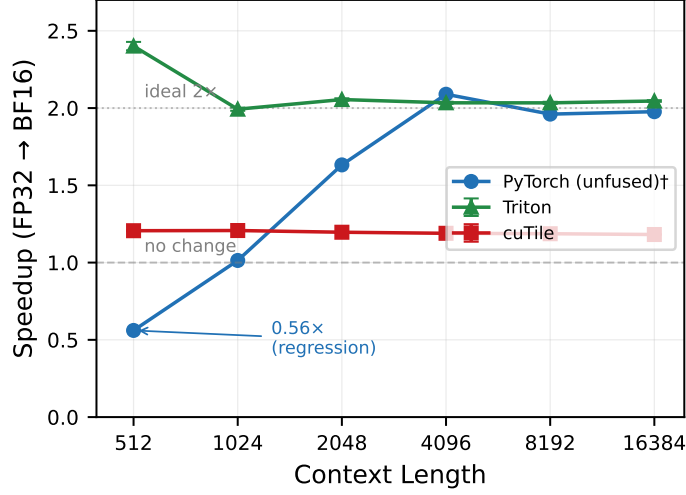


Fig. 3. Speedup from FP32 to BF16 for Paged Attention.

cuTile: $\sim 1.2\times$ (*baseline, limited*). The *baseline* cuTile kernel plateaus at 1.18–1.21 $\times$, remarkably consistent across all context lengths. The shared-memory staging and index-management overhead on the original `ct.gather` path is largely precision-independent: these costs dominate over the data-movement savings. This confirms our occupancy-based diagnosis for the baseline kernel. Critically, in BF16 this baseline cuTile implementation is *slower* than the unfused PyTorch reference at $\text{CTX} \geq 2048$ (e.g., $5,603 \pm 2.6$ vs. $3,357 \mu\text{s}$ at $\text{CTX}=4096$), reversing the FP32 ordering. However, the optimized BF16 variants in Table 4 show that this limited scaling characterizes the original kernel rather than an unavoidable property of cuTile.

PyTorch: *regression at small context*. PyTorch’s unfused implementation shows an anomalous regression at small context lengths: at $\text{CTX}=512$, BF16 is $1.8\times$ slower than FP32 (2,202 vs. 1,235 μs). The PyTorch BF16 latency plateaus at $\sim 2,200 \mu\text{s}$ for $\text{CTX} \leq 2048$ before recovering the expected $\sim 2\times$ scaling at large context. This plateau arises because each of the $\sim 16,000$ per-iteration sub-operations (matmul, softmax, elementwise) dispatches through cuBLAS, which selects suboptimal WMMA kernels for BF16 at small matrix sizes.

5 Discussion

Our results reveal a clear divide between cuTile’s behavior on regular kernels and the fragility of untuned irregular kernels. CuTile’s tile abstraction maps naturally to TMA-based contiguous loads and tensor-core execution, making it effective for *regular, compute-bound* kernels: it matches cuBLAS within $1.3\times$ on

GEMM and stays close to Triton on Flash Attention. For these workloads, the abstraction cost is negligible.

The optimized Paged Attention results substantially change the interpretation of the original negative result. The baseline cuTile kernel interacts poorly with `ct.gather`, auxiliary index-tensor loads, and conservative scheduling, producing the large gap shown in Tables 3 and 7. Yet a tuned kernel that eliminates those extra costs is $4.95\times$ faster than the original at CTX=4096 in BF16, and adding occupancy hints reduces the remaining gap to just 3.3% versus Triton. Because the optimized kernel still operates on the same paged, non-contiguous KV-cache layout, the baseline result cannot be interpreted as an intrinsic limitation of cuTile. Instead, the evidence points to a more nuanced conclusion: irregular kernels in cuTile are highly sensitive to implementation style and compiler guidance.

Our TileGym analysis remains informative, but should now be read more narrowly. Nvidia’s decode-attention kernels avoid `ct.gather` by assuming contiguous KV caches, and Paged Attention remains absent from TileGym. The specialized irregular-access benchmark shows that cuTile `ct.gather` is still slower than Triton’s block-pointer/TMA path, but only by $1.16\times$. This suggests that indirect access is a real sensitivity point, not a categorical blocker.

Warp’s position. Warp trails cuTile and cuBLAS by 3–4 $\times$ on GEMM and does not support BF16. Its primary strength is differentiable simulation; our results confirm it is not competitive for transformer inference.

Threats to validity. All experiments use a single consumer GPU. Data-center Blackwell GPUs (B100/B200) with more SMs could shift occupancy-sensitive results. We do not evaluate multi-GPU configurations or batch sizes representative of production serving. All reported measurements exhibit coefficients of variation below 1.1%, with the majority below 0.3%, indicating high reproducibility on our hardware.

6 Conclusion

We have presented one of the first comparative evaluations of Nvidia’s cuTile against Triton, Warp, and PyTorch across three transformer kernel primitives on Blackwell hardware. CuTile achieves within $1.3\times$ of cuBLAS on GEMM and stays close to Triton on Flash Attention, demonstrating that its tile abstraction incurs little overhead for regular, compute-bound workloads. For Paged Attention, the baseline cuTile kernel underperforms badly, but the optimized variant study shows that this is not the final takeaway: a tuned cuTile implementation improves BF16 latency at CTX=4096 by $9.05\times$ over the original and comes within 3.3% of Triton while preserving the same paged, non-contiguous KV-cache layout. A specialized indirect tile-extraction benchmark still leaves cuTile `ct.gather` $1.16\times$ slower than Triton’s block-pointer/TMA path, indicating that irregular access remains a real but manageable sensitivity. The key lesson is

therefore more nuanced than our initial baseline suggested: fair evaluation of cuTile on irregular workloads requires representative, tuned kernels rather than naive first implementations.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
2. Tillet, P., Kung, H.T., Cox, D.: Triton: An intermediate language and compiler for tiled neural network computations. In: *Proceedings of the 3rd MLSys Conference* (2019)
3. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C.: FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In: *Advances in Neural Information Processing Systems*, vol. 35 (2022)
4. Dao, T.: FlashAttention-2: Faster attention with better parallelism and work partitioning. In: *International Conference on Learning Representations* (2024)
5. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626 (2023)
6. Nvidia repositories: Warp, TileGym, cutlass, cutle-python. <https://github.com/NVIDIA/>
7. Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM* **52**(4), 65–76 (2009)
8. Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., Guestrin, C., Krishnamurthy, A.: TVM: An automated end-to-end optimizing compiler for deep learning. In: *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594 (2018)
9. Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., Amarasinghe, S.: Halide: A language and compiler for optimizing image processing pipelines. In: *ACM SIGPLAN Notices*, vol. 48, no. 6, pp. 519–530 (2013)
10. Jia, Z., Maggioni, M., Staiger, B., Scarpazza, D.P.: Dissecting the NVIDIA Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826* (2018)
11. Jia, Z., Maggioni, M., Smith, J., Scarpazza, D.P.: Dissecting the NVIDIA Ampere architecture via microbenchmarking. *arXiv preprint arXiv:2107.08569* (2021)
12. Reddi, V.J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., Wu, C.J., et al.: MLPerf inference benchmark. In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446–459. IEEE (2020)
13. Roesch, J., Lyubomirsky, S., Weber, L., Sevegnani, J., Smith, C., Tatlock, Z., Chen, T.: Relay: A new IR for machine learning frameworks. In: *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pp. 58–68 (2018)

14. Dao, T., Gu, A., et al.: Simplifying GPU programming with Triton. *IEEE Micro*, vol. 42, no. 4, pp. 64–72 (2022)
15. Aminabadi, R.Y., Black, S., Ammar, T., Metz, L., et al.: DeepSpeed-Inference: Enabling highly efficient generative applications. *arXiv preprint arXiv:2207.00032* (2022)
16. Shah, J., Dao, T., et al.: FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608* (2024)
17. Ye, Z., et al.: FlashInfer: Fast large language model inference on GPUs. *arXiv preprint arXiv:2403.00800* (2024)